

Clock Synchronization & Ordering

In a distributed system, time is a massive headache because there is no single "universal clock." If Server A says it received a request at 10:00:01 and Server B says 10:00:02, you can't be sure which happened first unless their clocks are perfectly in sync.

Physical Clocks (Real-World Time)

These measure actual seconds, minutes, and hours. We use them for things humans care about.

- **Mechanism:** Every computer has a quartz crystal clock, but they "drift" (gain or lose seconds). To fix this, we use **NTP (Network Time Protocol)**, which periodically syncs the computer's clock with a highly accurate atomic clock via the internet.
- **Best For:**
- **Logs:** "At what time did the server crash?"
- **Auditing:** "When did the user make this bank transfer?"
- **The Problem:** In high-speed systems, NTP isn't precise enough. If two events happen within milliseconds, physical clocks might record them in the wrong order due to network lag during the sync.

2. Logical Clocks (Event-Based Time)

Logical clocks don't care about the time of day. They only care about **Causality**: "Did Event A happen before Event B?" They use counters that increment whenever an event occurs.

Lamport Timestamps

This is a simple integer (a counter) that every node maintains.

- **The Rule:** Whenever a node does something, it increments its counter. When it sends a message, it includes that counter. When a receiver gets a message, it updates its own counter to be higher than the one it just received.
- **Result:** It provides a **Total Ordering**. You can say "Event 1 happened before Event 2."
- **Limitation:** If you see two timestamps, you can't tell if they were truly related (causal) or just happened at the same time on different servers.

Example: Ordering Events with Lamport Timestamps

Consider two nodes **A** and **B**, each maintaining a **logical counter**.

Step	Event	Timestamp
1	Initial state	A=0, B=0
2	A performs an event	A=1
3	A sends a message to B	Message carries timestamp 1
4	B receives the message	$B = \max(B, 1) + 1 \rightarrow \mathbf{2}$
5	B performs another event	B=3

Event timeline:

Event	Node	Timestamp
e1	A local event	1
e2	B receives message	2
e3	B local event	3

Because timestamps increase monotonically:

e1 → e2 → e3

The system can say **Event e1 happened before e2**, and **e2 before e3**.

💡 Limitation Example

Suppose two independent events occur:

Node	Event	Timestamp
A	update	4
B	update	5

Lamport timestamps say:

4 < 5 → A happened before B

But in reality the events may have been **independent** (no message exchange).

→ Lamport timestamps provide **total ordering**, but **cannot detect concurrency**.

This limitation is why distributed systems often use **vector clocks** when they need to detect **causal relationships and conflicts**.

Vector Clocks

This is a more advanced version where each node keeps an **array (vector)** of counters—one for every node in the system.

- **The Rule:** Node A knows its own counter AND its best guess of the counters for Node B and Node C. Every message carries this entire "vector" of information.
- **The Magic:** By comparing two vectors, the system can detect **Conflicts** or **Parallelism**.
- If Vector A is strictly "less than" Vector B, then A happened before B.
- If they are "mixed" (some numbers higher, some lower), the events happened **Concurrently** (at the same time).

Example: Detecting Causality with Vector Clocks

Consider a system with **three nodes: A, B, C**. Each node keeps a vector [A, B, C] representing its knowledge of events.

Event	Node	Vector Clock
Initial state	All nodes	[0,0,0]
A performs an event	A	[1,0,0]
A sends a message to B	B merges and increments	[1,1,0]
B performs another event	B	[1,2,0]

Now suppose **A and B update the same object independently**:

Node	Vector
A update	[2,1,0]
B update	[1,3,0]

Comparison:

- [2,1,0] vs [1,3,0]
- Some components are **larger in A**, others **larger in B**

→ The events are **concurrent**.

Conclusion: The system detects a **conflict**, meaning both updates happened independently and may require **merging or resolution**.

💡 Key intuition

Vector clocks allow distributed systems to distinguish:

Case	Meaning
A < B	Event A happened before B
B < A	Event B happened before A
Neither	Events happened concurrently